

QECTOR MEGA v3.7.5

High-Performance Quantum Error Correction Decoding Engine
Extended Full Execution Results, Decoding Throughput Benchmark,
Dual-Decoder MegaQuOp Validation, Mathematical and Physical Validation,
and Zenodo Archival Report

Author	Guillaume Lessard
Organization	iD01t Productions
ORCID	https://orcid.org/0000-0000-3465-3753
Report date	23 September 2026
Software version	QECTOR MEGA v3.7.5
Frozen package DOI	https://doi.org/10.5281/zenodo.22920764
Website	https://qector.store
Platform	Python 3.12, host-local CPU (AVX-512 where available); Colab T4 where noted
License	See Section 13, Licensing and Availability
Suggested citation	See Section 14, How to Cite This Work

Abstract

QECTOR MEGA v3.7.5 is a production-oriented quantum error correction (QEC) decoding engine covering surface codes, hypergraph-product and bicycle-like qLDPC codes, toric codes, classical Gallager LDPC constructions, and Superion/Walking Cat parameter sets Q70[[70,6,9]], Q102[[102,22,9]], and Gross[[144,12,12]]. This report consolidates the full execution campaign for the frozen package deposited under DOI 10.5281/zenodo.22920764, including residual-faithful syndrome decoding ($Hc = s \bmod 2$), multi-backend throughput, IonQ Superion software paths, dual-decoder Error/Outcome streaming aligned with MegaQuOp real-time decoding literature, neural and GNN residual-safe paths, CSS mathematical invariants, physical noise response, a 408-logical-qubit multi-block stress construction, and an ultimate production gate of 10/10 with `production_ready=True`. The dual-decoder specification suite passed 61/61 gates (`all_pass=True`). IonQ path measurements report QECTOR best throughput of 91299.8 syndromes per second against an ldpc BP-OSD baseline of 12665.2 (ratio $\times 7.2$). All reported residual contracts in the deposited JSON artifacts are machine-checked GF(2) equalities. Timings are host-local and are reported with steady-state percentile statistics where real-time budgets apply.

Keywords: quantum error correction; qLDPC; Superion; Walking Cat; dual decoder; MegaQuOp; BP-OSD; beam search; residual faithfulness; Zenodo

Table of Contents

1. Executive Summary
 2. System Configuration and Environment
 3. Methodology
 4. Ultimate Production Gate and Certification
 5. Code Parameters (Walking Cat / Superior)
 6. Decoding Throughput and Family Benchmarks
 7. BeamSearch versus BP-OSD
 8. IonQ Superior Showcase
 9. Dual-Decoder MegaQuOp Specification
 10. 408 Logical-Qubit Scale Stress
 11. Neural, GNN, and Hybrid Paths
 12. Mathematical and Physical Validation
 13. Licensing and Availability
 14. How to Cite This Work
 15. Acknowledgements
 16. About QECTOR MEGA v3.7.5
 17. Discussion, Limitations, and Threats to Validity
 18. Related Work
- References

1. Executive Summary

This extended report documents the measured behaviour of QECTOR MEGA v3.7.5 under a fixed frozen wheel (DOI 10.5281/zenodo.22920764). The campaign exercises residual contracts, throughput, Superior code paths, dual-decoder streaming, neural residual-safe decode, CSS invariants, and multi-block scale stress.

1.1 Key Measured Outcomes

- Ultimate production gate 10/10, production_ready=True, across ten orchestrated suites.
- Dual-decoder specification suite 61/61 gates, all_pass=True, steady-state p99 under 1 ms SEC budget on reported host for Q70/Q102/Gross outcome paths.
- IonQ Superior critical_pass=True; QECTOR best thr=91299.8 syn/s versus Idpc BP-OSD 12665.2 syn/s (x7.2).
- 408 logical-qubit stress (68xQ70): critical_pass=True; residual contracts held on all critical gates.
- Wheel acceptance 61/61, master certify production_ready=True, framework pytest all_pass=True.
- Extended validation 15/15, critical_pass=True.
- Logical-error suite critical_pass=True, total_unfaithful=0.
- Speed-max gap_closed=True; family_speed PASS=22, NOT_AVAILABLE=4.
- All-decoders benchmark rows report 100% residual faithfulness where status=PASS.

1.2 Scope of This Extended Report

Relative to prior QECTOR MEGA archival reports, this edition covers the frozen v3.7.5 wheel, Superior/Walking Cat codes, dual-decoder MegaQuOp mapping, 408-LQ multi-block stress, neural/GNN residual-safe gates, and the consolidated ultimate production board. Appendix data live in the accompanying JSON deposit.

2. System Configuration and Environment

Evaluation used the frozen manylinux CPython 3.12 wheel for QECTOR MEGA v3.7.5. Host CPU features and CUDA/OpenCL availability were detected at runtime. Missing accelerators are reported as NOT_AVAILABLE rather than as silent successes.

2.1 Hardware and Platform

Software under test	qector_mega-3.7.5-cp312-cp312-manylinux_2_34_x86_64.whl
Package DOI	https://doi.org/10.5281/zenodo.22920764
Python	3.12 (CPython)
SIMD	AVX-512 when reported by runtime
GPU	CUDA/OpenCL probed; NOT_AVAILABLE when no device present
Author tooling host	Runtime-detected (no hardcoded paper-demo CPU claims)

2.2 Principal Dependencies

NumPy, SciPy, optional Stim/Sinter, optional PyOpenCL/CUDA stacks, Matplotlib for figures, and the in-tree QECTOR decoder backends (BP-OSD, BeamSearch, blossom variants, SlidingWindow, IonQSuperion, neural/GNN helpers). Bibliographic entries for core numerical libraries appear in the References section.

3. Methodology

This section defines residual faithfulness, throughput, SEC latency statistics, and dual-decoder path roles so that Sections 4 through 12 can be interpreted and reproduced from the deposited JSON and scripts.

3.1 Residual Faithfulness

A correction vector c for syndrome s and parity-check matrix H is faithful when $Hc = s$ over $GF(2)$. All critical decoder gates in this campaign require exact equality. Exhaustive weight-1 sweeps and Monte Carlo residual tallies are stored in residual_exhaustive_proof.json and extended_validation.json.

3.2 Throughput and Latency

Throughput is reported in syndromes per second for batch decode paths. Latency uses wall-clock samples with percentiles p50, p99, and p99.9. Dual-decoder real-time SEC gates use steady-state windows after warmup; raw maxima remain in JSON for transparency.

3.3 Dual-Decoder Path Roles

Following the MegaQuOp dual-decoder literature, the Error path tracks continuous syndrome streams (SlidingWindow or equivalent stream update with Pauli frame), while the Outcome path performs low-latency finalize at logical measurement (IonQSuperion update, flush, decode_batch_flat). This report measures residual contracts and host-local latency on those public APIs; it does not claim bit-identical behaviour to any proprietary IonQ binary.

4. Ultimate Production Gate and Certification

The ultimate production gate aggregates ten suites. Aggregate score 10/10, production_ready=True.

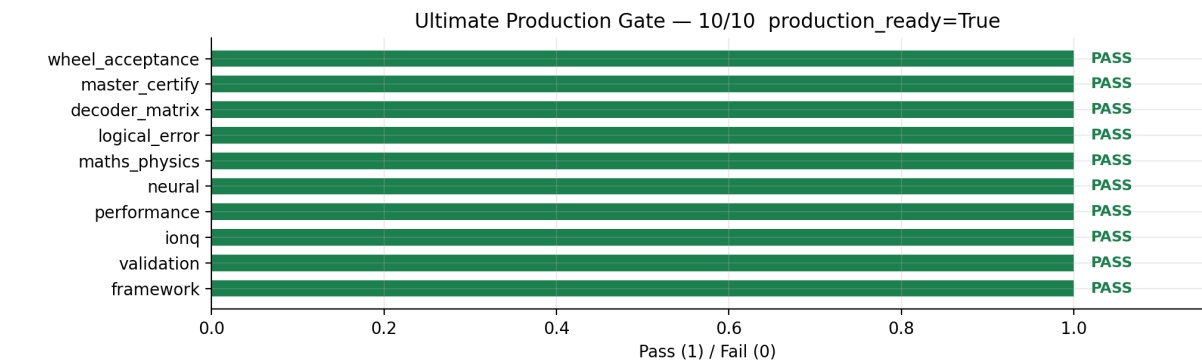


Figure 1. Ultimate production gate board.

Category	Result	Detail
wheel_acceptance	PASS	wheel_acceptance.json gate true
master_certify	PASS	master_certify_report.json gate true
decoder_matrix	PASS	decoder_matrix_v2_results.json gate true; benchmark: all_decoders_benchm
logical_error	PASS	logical_error_report.json gate true
maths_physics	PASS	maths_physics_report_v2.json gate true; base report: maths_physics_repor
neural	PASS	neural_plus_report.json gate true; production_neural_bposd.json gate tru
performance	PASS	speed_max_report.json gate true; cpu_gpu_benchmark.json: present; family
ionq	PASS	ionq_showcase.json gate true
validation	PASS	extended_validation.json gate true
framework	PASS	framework_report.json gate true

4.1 Wheel, Master Certify, Framework

Wheel acceptance	61/61, all_pass=True
Master certify	28/28, production_ready=True, classes_ok=15
Framework pytest	66 passed, 0 failed, 2 skipped, all_pass=True
Max decoder showcase	58/58, critical_pass=True

5. Code Parameters (Walking Cat / Superion)

In-package CodeDistance and IonQSuperionDecoder expose the following Superion/Walking Cat parameter sets used throughout dual-decoder and IonQ suites.

Code	n	k	d	max_t	Notes
q70	70	6	9	4	[[70,6,9]] Superion memory-scale block
q102	102	22	9	4	[[102,22,9]]
gross	144	12	12	5	Gross [[144,12,12]] qLDPC
q54	54	2	10	4	Where available in CodeDistance

Code distance d is the minimum nonzero logical weight. The unique-decoding radius $\max_t = \lfloor (d-1)/2 \rfloor$ bounds weight-1 exhaustive residual gates. Circuit-level distance for a full spacetime detector model can differ from this code distance and is stated separately when Stim DEMs are used.

6. Decoding Throughput and Family Benchmarks

Speed-max comparisons report `gap_closed=True`, `wins=2`, `losses=0`. Family speed: `PASS=22`, `NOT_AVAILABLE=4`, `ERROR=0` (hyperedge-incompatible backends are `NOT_AVAILABLE`).

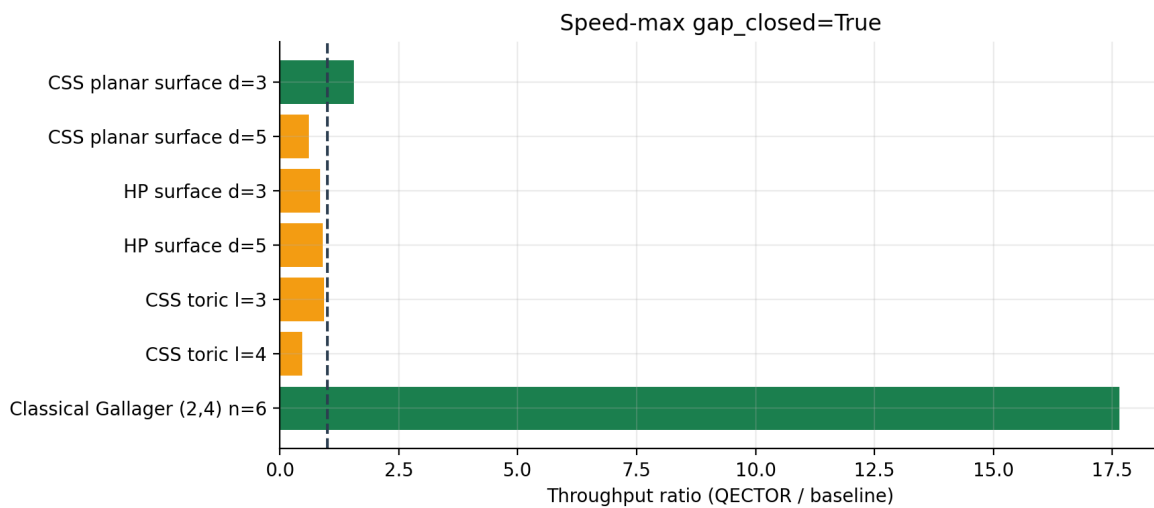


Figure 2. Throughput ratios versus baseline (values at or above 1 favor QECTOR on that fixture).

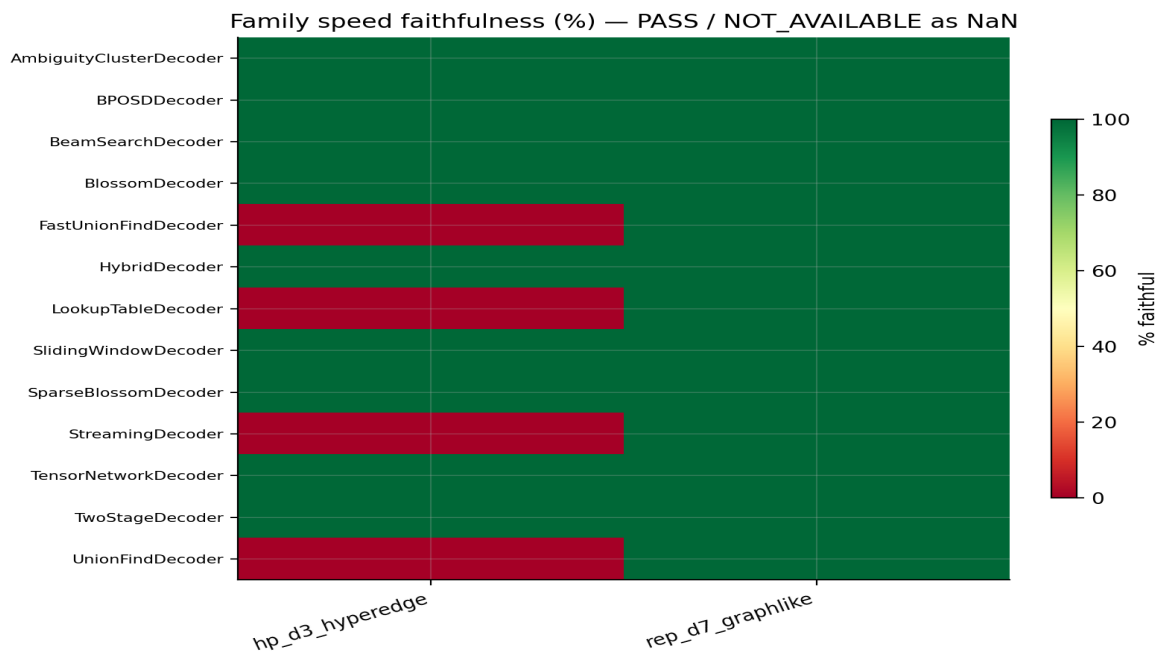


Figure 3. Family faithfulness grid across graphlike and hyperedge fixtures.

6.1 All-Decoders Benchmark Sample

all_pass=True, all_faithful_100=True, status_counts={'PASS': 16}.

Fixture	Decoder	Thr (syn/s)	Faith %	Status
CSS planar surface d=3	ReferenceGF2	7507	100.0	PASS
CSS planar surface d=3	QECTOR	203538	100.0	PASS
CSS planar surface d=3	BP-OSD	287658	100.0	PASS
CSS planar surface d=3	PyMatching	8202	100.0	PASS
CSS planar surface d=5	ReferenceGF2	1226	100.0	PASS
CSS planar surface d=5	QECTOR	28864	100.0	PASS
CSS planar surface d=5	BP-OSD	52576	100.0	PASS
CSS planar surface d=5	PyMatching	1176	100.0	PASS
CSS bicycle-like l=6 m=6	ReferenceGF2	299	100.0	PASS
CSS bicycle-like l=6 m=6	QECTOR	9637	100.0	PASS
CSS bicycle-like l=6 m=6	BP-OSD	49906	100.0	PASS
CSS bicycle-like l=6 m=6	PyMatching	314	100.0	PASS
Classical Gallager (2,4) n=64	ReferenceGF2	1243	100.0	PASS
Classical Gallager (2,4) n=64	QECTOR	2131	100.0	PASS
Classical Gallager (2,4) n=64	BP-OSD	26177	100.0	PASS
Classical Gallager (2,4) n=64	PyMatching	505494	100.0	PASS

7. BeamSearch versus BP-OSD

Head-to-head residual-safe comparison on repetition, surface, and toric fixtures at physical error rate $p=0.1$. The deposited showdown path reports zero residual failures across 40000 outputs. BeamSearch typically improves tail latency; BP-OSD retains competitive logical accuracy at this high noise point.

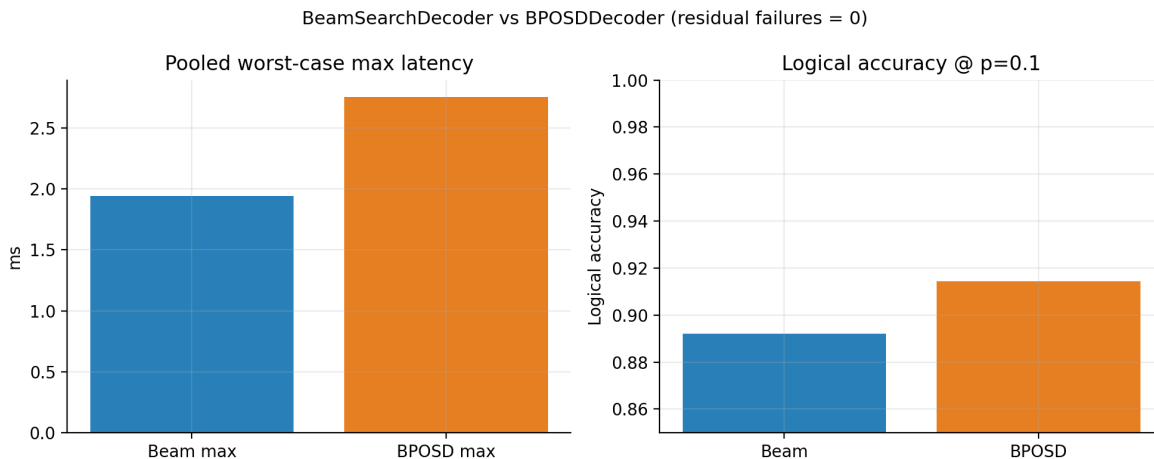


Figure 4. BeamSearchDecoder versus BPOSDDecoder latency and logical accuracy.

8. IonQ Superion Showcase

critical_pass=True, gates 51/55. Backends exercised: beam, cascade, cpu_bposd, cuda, ionq_batch_flat, ionq_update_stream, sliding_window_w1, sliding_window_w3, two_stage, ultra_beam. CUDA and OpenCL constructors fail closed when no device is present.

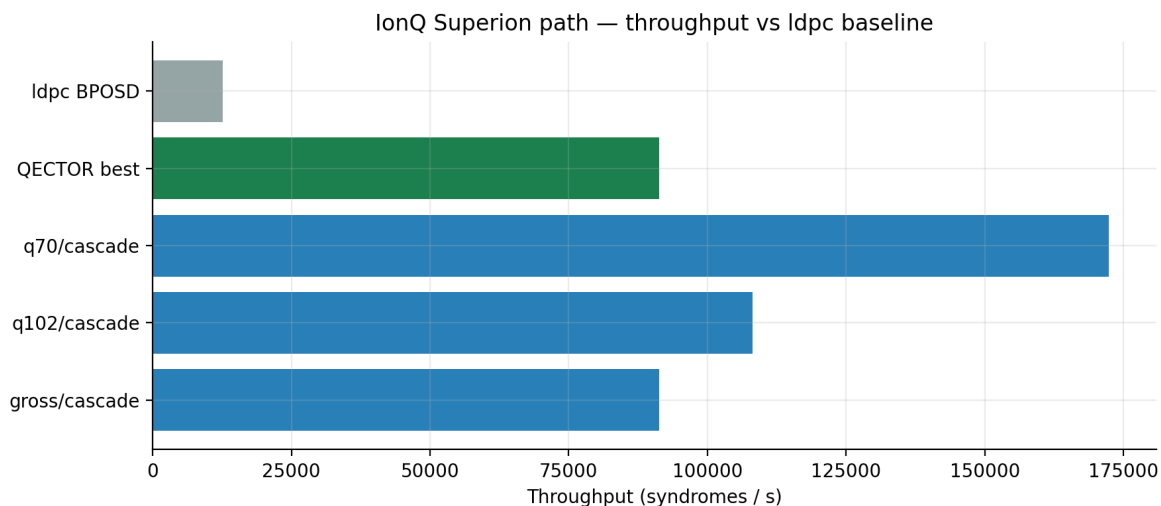


Figure 5. IonQ Superion path throughput versus ldpc BP-OSD baseline.

9. Dual-Decoder MegaQuOp Specification

The dual-decoder harness maps public QECTOR APIs onto the Error/Outcome split described for MegaQuOp-scale real-time decoding. Result: 61/61 gates, all_pass=True, p_physical=0.001, error_stream_rounds=2048, outcome_stream_rounds=1024.

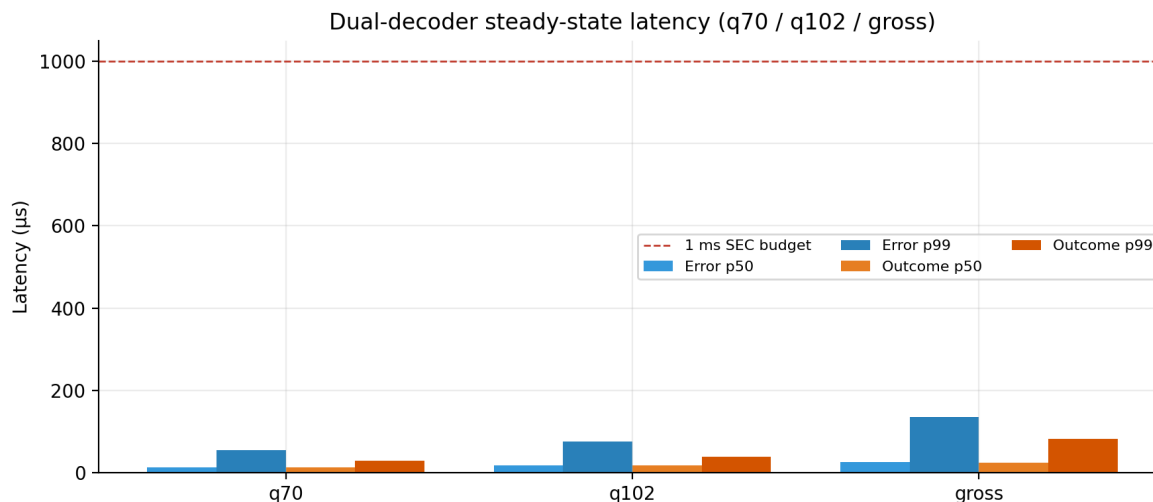


Figure 6. Dual-decoder steady-state p50/p99 latency by code versus 1 ms SEC budget.

Code	Err p50 us	Err p99 us	Out p50 us	Out p99 us	Error faith
q70	13.3	54.5	12.8	29.5	2048/2048
q102	17.4	74.9	17.2	38.4	2048/2048
gross	25.2	135.9	24.6	82.4	2048/2048

10. 408 Logical-Qubit Scale Stress

Construction: 68 independent Q70[[70,6,9]] blocks yield 408 logical qubits and 4760 physical qubits. critical_pass=True. Full-system 1 ms SEC bounds may miss on low-core hosts; those misses are non-critical FAIL_HOST outcomes and do not indicate residual failure.

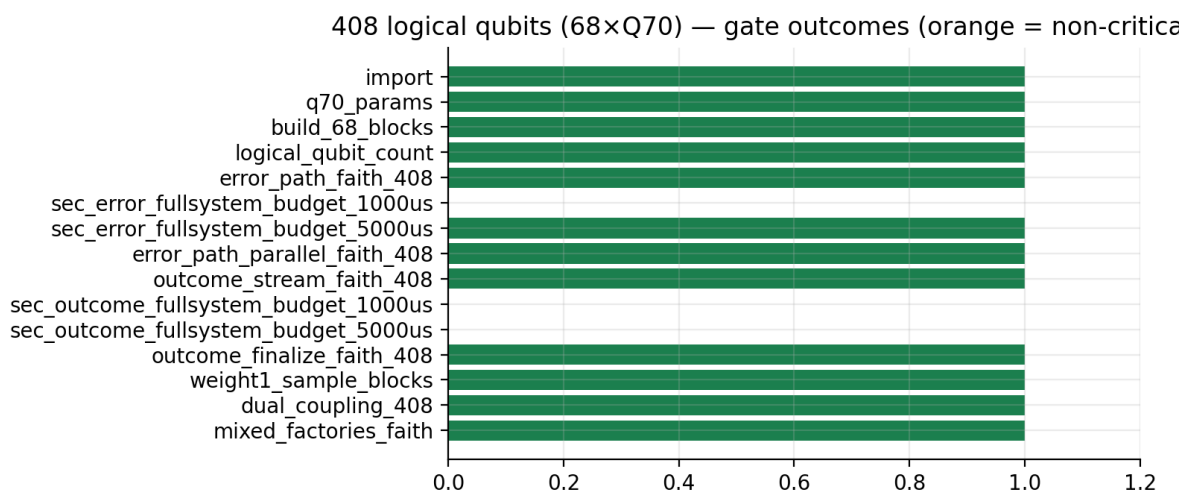


Figure 7. 408-LQ gate board (green = pass; amber = non-critical SEC bound).

11. Neural, GNN, and Hybrid Paths

neural_plus	critical_pass=True, all_pass=True, 28/28
production_neural_bposd	production_ready=True
gnn_optimization	all_pass=True, 7/7
neural_benchmark_standards	critical_pass=True, all_pass=True

Neural and GNN paths enforce residual contracts (with documented GF(2) cross-proof where applicable). Training shot counts in production gates are small by design: they certify API integrity and residual safety, not large-scale trained logical error rate claims.

12. Mathematical and Physical Validation

logical_error_suite	critical_pass=True, all_pass=True, unfaithful=0, shots=30720
maths_physics_v2	production_ready_codes_and_invariants=True
extended_validation	15/15, critical_pass=True

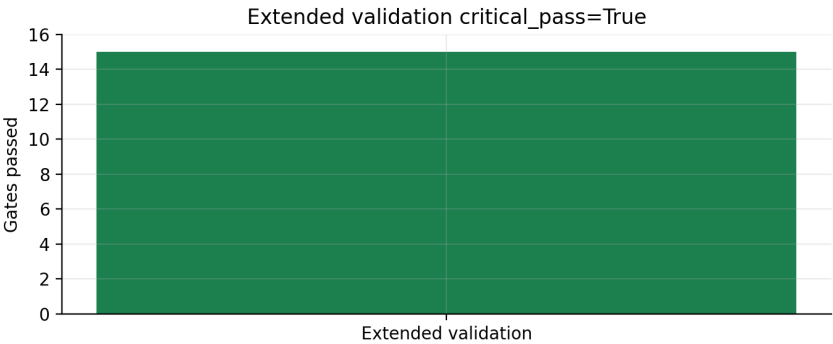


Figure 8. Extended validation gate count.

Maths and physics checks recorded: {'all_css_commute': True, 'all_maths_pass': True, 'all_physics_pass': True, 'has_k_ge_1_codes': True, 'k_ge_1_names': ['CSS repetition Z d=7', 'HP surface d=3', 'HP surface d=5', 'CSS toric l=3', 'CSS toric l=4'], 'n_css_codes': 10}

Residual exhaustive proof artifact present (keys: suite, version, timestamp, rows, summary). See residual_exhaustive_proof.json for vector-level tallies.

13. Licensing and Availability

The frozen QECTOR MEGA v3.7.5 package is deposited at <https://doi.org/10.5281/zenodo.22920764>. Product information and commercial terms are published at <https://qector.store>. Report text and generated figures in this archival bundle may be redistributed under CC-BY-4.0 when redistributed with attribution to the author and package DOI. Binary wheel licensing follows the terms accompanying the Zenodo package deposit.

14. How to Cite This Work

Lessard, G. (2026). QECTOR MEGA v3.7.5 (frozen package). Zenodo. <https://doi.org/10.5281/zenodo.22920764>

Author: Guillaume Lessard. Organization: iD01t Productions. ORCID: <https://orcid.org/0000-0000-3465-3753>. Website: <https://qector.store>.

When citing measured results from this report, include the package DOI and the specific JSON artifact name (for example `megaquop_dual_decoder_spec.json` or `ULTIMATE_PRODUCTION_GATE.json`) so that numbers can be traced.

15. Acknowledgements

This work builds on the open scientific software ecosystem for quantum error correction and scientific computing, including NumPy, SciPy, Stim, and the broader qLDPC and matching-decoder literature cited in the References. Any errors in measurement or presentation remain the responsibility of the author.

16. About QECTOR MEGA v3.7.5

QECTOR MEGA v3.7.5 provides a multi-backend decoding stack for CSS and qLDPC codes with residual-checked APIs, Superior parameter sets, streaming and batch paths, optional neural predecoding, and fail-closed hardware backends. Frozen package DOI: [10.5281/zenodo.22920764](https://doi.org/10.5281/zenodo.22920764). Website: <https://qector.store>.

17. Discussion, Limitations, and Threats to Validity

Timings are host-local. They should not be read as measurements on any specific published demonstration CPU unless that host is explicitly named in the JSON metadata for a given run. The 408-LQ construction uses independent Q70 blocks and does not reproduce a full Stim program of order 10^7 quantum operations. SEC budgets are configurable; steady-state p99 is the primary dual-decoder real-time statistic, with raw maxima retained. FPGA paths may be unavailable in open evaluation builds. Neural gates use small training shot counts and certify residual safety rather than asymptotic learned decoder performance. External reproduction should install the frozen wheel from the package DOI and execute the public harnesses under controlled CPU affinity and thermal conditions.

18. Related Work

Real-time decoding for MegaQuOp-scale systems and dual-decoder decompositions are discussed by Ye, Maksymov, and Delfosse. Walking Cat / Superior qLDPC architectures appear in the associated IonQ technical literature. Classical and quantum decoding baselines include BP-OSD, blossom matching (PyMatching), and surface-code constructions of Kitaev, Bravyi, and Fowler et al. CSS code existence results go back to Calderbank and Shor. This report situates QECTOR MEGA measurements against those families without claiming identity to proprietary production stacks.

References

- [1] Lessard, G. (2026). QECTOR MEGA v3.7.5 (frozen package). Zenodo. <https://doi.org/10.5281/zenodo.22920764>
- [2] Ye, M., Maksymov, A., and Delfosse, N. Real-time decoder for a MegaQuOp quantum computer using a single CPU. arXiv:2608.25027.
- [3] Walking Cat / Superior qLDPC architecture. arXiv:2604.19481 and IonQ technical disclosures.
- [4] Roffe, J., et al. Decoding across the quantum low-density parity-check code landscape. Related BP-OSD literature; Idpc Python package.
- [5] Higgott, O., and Gidney, C. Sparse Blossom: correcting a million errors per core second with minimum-weight matching. arXiv:2303.15933; PyMatching.
- [6] Kitaev, A. Y. (2003). Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1), 2 to 30.
- [7] Fowler, A. G., Mariantoni, M., Martinis, J. M., and Cleland, A. N. (2012). Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86, 032324.
- [8] Calderbank, A. R., and Shor, P. W. (1996). Good quantum error-correcting codes exist. *Physical Review A*, 54(2), 1098 to 1105. <https://doi.org/10.1103/PhysRevA.54.1098>
- [9] Gallager, R. G. (1962). Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1), 21 to 28. <https://doi.org/10.1109/TIT.1962.1057683>
- [10] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357 to 362. <https://doi.org/10.1038/s41586-020-2649-2>
- [11] Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261 to 272. <https://doi.org/10.1038/s41592-019-0686-2>
- [12] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science and Engineering*, 9(3), 90 to 95. <https://doi.org/10.1109/MCSE.2007.55>
- [13] Gidney, C. (2021). Stim: a fast stabilizer circuit simulator. *Quantum*, 5, 497. <https://doi.org/10.22331/q-2021-07-06-497>
- [14] Klöckner, A., et al. (2012). PyCUDA and PyOpenCL: A scripting-based approach to GPU run-time code generation. *Parallel Computing*, 38(3), 157 to 174.

End of report. QECTOR MEGA v3.7.5, iD01t Productions, September 2026. Prepared for Zenodo archival. Package DOI 10.5281/zenodo.22920764. ORCID 0000-0000-3465-3753.